

SERVERLESS ARCHITECTURE USING AWS FOR SMART OCR EXPENSE TRACKER

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr.Juttuka Lovababu, APSSDC

Submitted by

TUMMALA GEETHAMRUTHA, 23BQ1A61D9, VVIT NAMBUR, (Batch -1)

MEKATHOTI MADHURI, 23BQ1A1269, VVIT NAMBUR, (Batch -1)

PILLI MAHIMA, 23BQ1A1295, VVIT NAMBUR, (Batch -1)

PERLA SARALA, 23BQ1A1291, VVIT NAMBUR (Batch -1)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr Juttuka Lovababu, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Tummala Geethamrutha, 23BQ1A61D9, geethamrutha.tummala06@gmail.com.

Mekathoti Madhuri, 23BQ1A1269, madhurimekathoti0@gmail.com.

Pilli Mahima, 23BQ1A1295, pillimahima9@gmail.com.

Perla Sarala, 23BQ1A1291 , perlasarala38@gmail.com.

Nuthalapati Chaitanya Sri, 23BQ1A1280, nuthalapatichaitanyasrichowdar@gmail.com.

ABSTRACT

This project presents the implementation of a **The Smart OCR Expense Tracker** is a cloud-based, serverless application developed to simplify and automate expense management by leveraging the power of **Amazon Web Services (AWS)** and **Optical Character Recognition (OCR)** technology. The primary objective of this project is to eliminate manual data entry by automatically extracting expense information from receipt images and storing it securely for future reference and analysis.

The application allows users to upload receipts such as shopping bills, restaurant invoices, grocery bills, fuel receipts, hotel bills, and other purchase receipts through a user-friendly web interface. Once a receipt image is uploaded, it is securely stored in **Amazon S3**, which acts as the cloud storage service. The uploaded receipt triggers an **AWS Lambda** function that invokes **Amazon Textract** to perform OCR on the image. Amazon Textract accurately extracts important information such as the merchant name, purchase date, total amount, tax details, and other relevant expense data from the receipt.

After the text extraction process is completed, the backend processes and validates the extracted information before storing it in **Amazon DynamoDB**, a fully managed NoSQL database. **Amazon API Gateway** is used to establish secure communication between the frontend application and backend services, enabling users to access their expense records efficiently. The application provides users with a centralized dashboard where they can view, search, filter,

and manage all their recorded expenses. Additionally, users can monitor their monthly spending, review their expense history, and gain better insights into their financial habits.

The project follows a **serverless architecture**, which eliminates the need to provision or manage servers manually. This architecture provides high scalability, improved reliability, reduced infrastructure costs, and simplified deployment. Since AWS services automatically scale according to workload, the application remains efficient even when processing multiple receipts simultaneously.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION	5
2.	METHODOLOGY	6
3.	TECHNOLOGY STACK.....	10
4.	SYSTEM DESIGN / ARCHITECTURE	16
5.	IMPLEMENTATION	28
6.	RESULTS	33
7.	CONCLUSION	36

1. INTRODUCTION

In today's digital era, managing personal and business expenses efficiently has become increasingly important. People regularly receive receipts for purchases such as groceries, restaurant bills, fuel, hotel stays, shopping, and utility payments. Traditionally, these expenses are recorded manually, which is time-consuming, repetitive, and often results in inaccurate or incomplete records. As the number of receipts increases, maintaining an organized expense history becomes more difficult, making it challenging to monitor spending patterns and prepare financial reports.

The **Smart OCR Expense Tracker** is a cloud-based application developed to simplify and automate the process of expense management using **Amazon Web Services (AWS)** and **Optical Character Recognition (OCR)** technology. Instead of manually entering receipt information, users simply upload a receipt image through the application. The system automatically extracts relevant expense details such as the merchant name, purchase date, total amount, and other important information using **Amazon Textract**, AWS's intelligent OCR service.

The uploaded receipt images are securely stored in **Amazon S3**, while **AWS Lambda** performs the backend processing required to extract, validate, and organize the expense information. The processed data is then stored in **Amazon DynamoDB**, providing a secure and scalable database for maintaining expense records. **Amazon API Gateway** enables seamless communication between the frontend application and backend services, allowing users to easily access and manage their expense data.

The application provides users with a centralized dashboard where they can view, search, filter, and organize their expenses. It also helps users monitor monthly spending, maintain a complete digital record of receipts, and gain valuable insights into their financial habits. By automating receipt processing, the system significantly reduces manual effort, minimizes data entry errors, improves data accuracy, and saves valuable time.

One of the major advantages of this project is its **serverless architecture**, which eliminates the need for managing physical servers or complex infrastructure. AWS automatically handles resource provisioning, scaling, availability, and maintenance, making the application highly scalable, reliable, secure, and cost-effective. This architecture also enables the system to process multiple receipt uploads efficiently without requiring manual intervention.

The **Smart OCR Expense Tracker** demonstrates the practical application of cloud computing, OCR technology, serverless computing, and NoSQL databases in solving a real-world financial management problem. The project not only simplifies expense tracking for individuals and businesses but also showcases how modern AWS cloud services can be integrated to develop intelligent, scalable, and automated applications. It serves as an excellent example of combining artificial intelligence with cloud computing to build practical solutions that improve productivity, accuracy, and overall user experience.

2. METHODOLOGY

The development of the **Smart OCR Expense Tracker** followed a structured methodology to ensure an efficient, secure, and scalable expense management solution. The project was developed through multiple phases, including requirement analysis, system design, implementation, testing, deployment, and continuous improvement. Each phase focused on building a reliable serverless application using AWS cloud services. The methodology adopted is described below.

2.1 Requirement Analysis

The first phase involved identifying the problems associated with traditional expense management systems. Manual entry of receipt information is time-consuming, prone to errors, and difficult to manage when users have multiple receipts. Based on these challenges, the following objectives were defined:

- Automate receipt data extraction.
- Eliminate manual expense entry.
- Securely store expense records.
- Provide users with expense history and spending analysis.
- Build a scalable serverless cloud application using AWS.

The functional and technical requirements were analyzed to select suitable AWS services for the project.

2.2 System Design

Based on the identified requirements, a serverless architecture was designed using Amazon Web Services (AWS). The system consists of a frontend application for uploading receipt images and backend AWS services that process and store expense data.

The workflow begins when a user uploads a receipt image through the application. The image is stored in Amazon S3, which automatically triggers an AWS Lambda function. Lambda invokes Amazon Textract to perform Optical Character Recognition (OCR) and extract the receipt information. The extracted data is validated, processed, and stored in Amazon

DynamoDB. Finally, users can access their expense records through the application dashboard.

The architecture was designed to ensure scalability, high availability, security, and low operational cost.

2.3 Implementation

The implementation phase involved integrating multiple AWS services to automate the expense tracking process.

Users upload receipt images through the frontend interface. Amazon S3 stores the uploaded images and generates an event notification whenever a new receipt is uploaded. This event automatically triggers an AWS Lambda function.

The Lambda function calls Amazon Textract, which extracts important information such as:

- Merchant Name
- Purchase Date
- Total Amount
- Tax Information
- Other available receipt details

The extracted information is then processed and formatted before being stored in Amazon DynamoDB. Amazon API Gateway enables secure communication between the frontend application and backend services, allowing users to retrieve and manage their expense records.

2.4 Database Management

Amazon DynamoDB is used as the primary database because it provides fast, scalable, and fully managed NoSQL storage.

Each expense record contains information such as:

- Receipt ID
- Merchant Name
- Purchase Date
- Total Amount
- Receipt Category
- Receipt Image Reference
- Upload Timestamp

The database allows users to efficiently search, retrieve, and organize their expense history while supporting high-speed read and write operations.

2.5 Testing

Several testing techniques were performed to ensure that every component of the application functions correctly.

The receipt upload process was tested to verify successful image storage in Amazon S3.

OCR accuracy was validated by testing various receipt formats generated from grocery stores, restaurants, fuel stations, shopping malls, and hotel invoices.

The extracted data was compared with the original receipt to verify correctness before storing it in DynamoDB.

Integration testing ensured smooth communication among Amazon S3, AWS Lambda, Amazon Textract, API Gateway, and DynamoDB. The application was also tested for invalid images, incomplete receipts, and unsupported file formats to ensure proper error handling.

2.6 Deployment

The complete application was deployed on the AWS Cloud using a serverless architecture.

The deployment included:

- Amazon S3 for receipt image storage
- AWS Lambda for backend processing
- Amazon Textract for OCR
- Amazon DynamoDB for database storage
- Amazon API Gateway for REST API communication
- AWS IAM for secure access management
- Amazon CloudWatch for monitoring and logging

CloudWatch continuously monitors Lambda executions and system performance, making debugging and maintenance easier.

2.7 Continuous Improvement

The project was developed using an iterative approach, allowing continuous improvements throughout development.

Several enhancements were identified for future versions, including:

- Automatic expense categorization using Artificial Intelligence.
- Monthly budget planning and spending alerts.
- Interactive dashboards and graphical expense analytics.
- Multi-user authentication using Amazon Cognito.
- Exporting expense reports to PDF or CSV.

- Mobile application integration for Android and iOS devices.

The modular serverless architecture makes these enhancements easy to integrate without major modifications to the existing system.

The methodology adopted for the **Smart OCR Expense Tracker** ensures a secure, scalable, and fully automated expense management solution. By integrating Amazon S3, Amazon Textract, AWS Lambda, Amazon DynamoDB, API Gateway, IAM, and CloudWatch, the system successfully automates receipt processing, minimizes manual effort, improves data accuracy, and provides users with an efficient platform for tracking and managing their expenses.

3. TECHNOLOGY STACK

The **Smart OCR Expense Tracker** was developed using a modern cloud-based technology stack that combines serverless computing, Optical Character Recognition (OCR), cloud storage, and database technologies. The application leverages Amazon Web Services (AWS) to provide a scalable, secure, and cost-effective solution for automating expense management. The following technologies and services were used throughout the development of the project.

3.1 Cloud Platform

Amazon Web Services (AWS)

Amazon Web Services (AWS) serves as the primary cloud platform for the application. It provides fully managed services that eliminate the need for server management while ensuring high availability, scalability, security, and reliability.

3.2 AWS Services

Amazon S3 (Simple Storage Service)

Amazon S3 is used to securely store uploaded receipt images. It acts as the entry point of the application and automatically triggers backend processing whenever a new receipt is uploaded.

Features:

- Secure cloud storage
 - High durability and availability
 - Automatic event notifications
 - Supports image and document storage
-

Amazon Textract

Amazon Textract is the core OCR service used in the project. It extracts printed text and important receipt information such as merchant name, purchase date, total amount, taxes, and other expense details from uploaded receipt images.

Features:

- Optical Character Recognition (OCR)
 - Automatic receipt data extraction
 - High accuracy for printed documents
 - Supports images and PDF files
-

AWS Lambda

AWS Lambda provides serverless backend processing. It automatically executes the application logic whenever a receipt image is uploaded to Amazon S3.

The Lambda function:

- Receives S3 upload events
- Calls Amazon Textract
- Processes extracted data

- Stores expense records in DynamoDB
 - Returns processed data to the application
-

Amazon DynamoDB

Amazon DynamoDB is a fully managed NoSQL database used to store expense records extracted from receipts.

Each record contains:

- Receipt ID
- Merchant Name
- Purchase Date
- Total Amount
- Receipt Details
- Upload Timestamp

Features:

- High-speed data retrieval
 - Automatic scaling
 - Low latency
 - Fully managed database service
-

Amazon API Gateway

Amazon API Gateway provides secure REST APIs that connect the frontend application with backend AWS services.

Functions:

- API management
 - Secure communication
 - Request routing
 - Integration with AWS Lambda
-

AWS IAM (Identity and Access Management)

AWS IAM manages authentication and permissions between AWS services.

IAM roles ensure:

- Secure access to Amazon S3
 - Permission to invoke Lambda
 - Controlled access to DynamoDB
 - Secure interaction with Textract
-

Amazon CloudWatch

Amazon CloudWatch monitors the overall system and stores logs generated by AWS Lambda.

It is used for:

- Monitoring Lambda execution
 - Viewing application logs
 - Debugging errors
 - Performance monitoring
-

3.3 Frontend Technologies

The user interface was developed using modern web technologies to provide an easy-to-use expense management system.

- **React.js** – Builds an interactive and responsive user interface.
 - **HTML5** – Structures the web pages.
 - **CSS3** – Designs and styles the application.
 - **JavaScript (ES6)** – Implements client-side functionality and API communication.
-

3.4 Backend Technologies

The backend handles business logic, API requests, and AWS service integration.

- **Python 3.x** – Used to implement backend logic.
 - **Flask Framework** – Provides RESTful APIs and integrates with AWS services.
 - **Boto3 SDK** – Python SDK used to interact with AWS services such as S3, Textract, Lambda, and DynamoDB.
-
-

3.5 Database

Amazon DynamoDB

A NoSQL cloud database used for storing extracted expense information with fast and scalable read/write operations.

3.6 Data Format

The application processes multiple data formats during expense tracking.

- **JPEG (.jpg)**
 - **PNG (.png)**
 - **PDF (.pdf)** *(optional for future enhancement)*
 - **JSON** (API communication and data exchange)
-

3.7 Security

The project follows AWS security best practices.

- AWS IAM Roles and Policies
 - Secure API communication
 - Cloud-based encrypted storage
 - Controlled service permissions
 - Secure access to backend resources
-
-

3.8 Monitoring and Logging

The application uses AWS monitoring services to improve reliability.

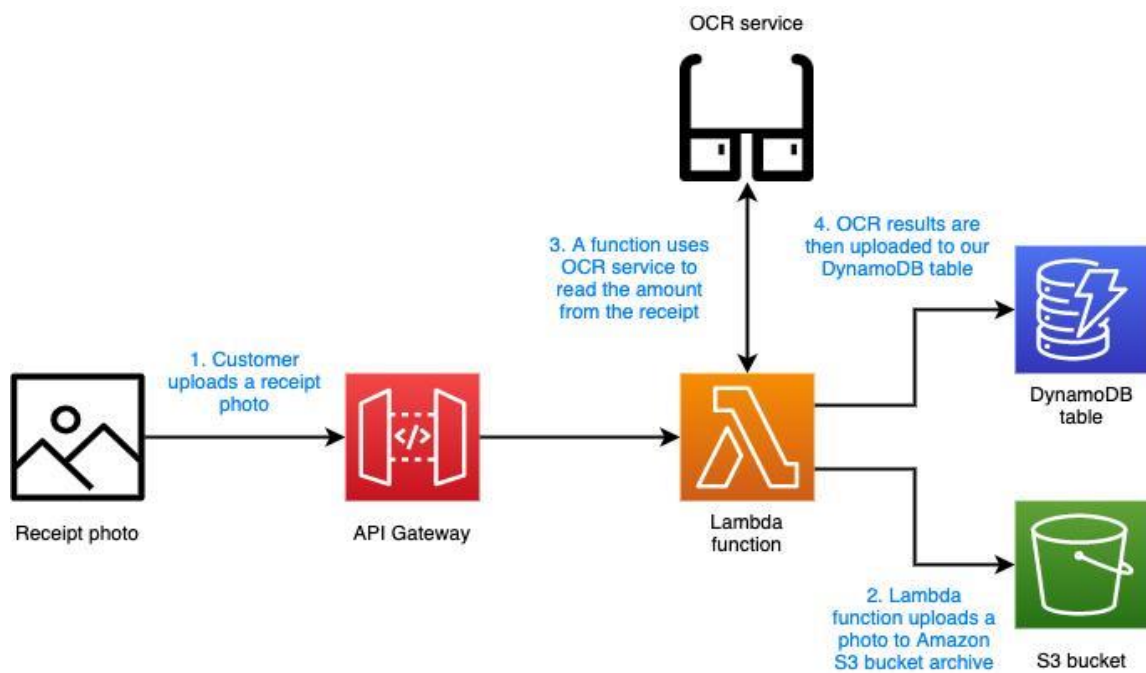
- **Amazon CloudWatch Logs** – Records Lambda execution logs.
- **CloudWatch Metrics** – Monitors application performance, errors, and execution time.

3. SYSTEM DESIGN / ARCHITECTURE

The **Smart OCR Expense Tracker** is designed as a cloud-based, serverless application that automates receipt processing and expense management. The system leverages Amazon Web Services (AWS) to provide a scalable, reliable, and secure architecture while eliminating the need for traditional server management. The application follows an event-driven workflow where receipt images uploaded by users are automatically processed using Optical Character Recognition (OCR) technology, and the extracted expense information is stored securely for future retrieval and analysis.

The architecture consists of multiple layers, including the user interface, storage layer, processing layer, OCR engine, database layer, and monitoring services. Each AWS service performs a specific function within the system, ensuring efficient data processing and seamless communication between components.

The overall architecture is illustrated through the following functional components.



4.1 User Interface

The application provides a simple and user-friendly web interface developed using **React.js**. Users can upload receipt images directly from their computer or mobile device. The frontend also displays the extracted expense information, expense history, monthly spending summaries, and search functionality.

Functions

- Upload receipt images
- View extracted expense details
- Search previous expenses
- View monthly expense reports
- Manage expense records

Aurora and RDS > Databases > Create database

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

expense-tracker-db

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username [Info](#)
Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management

You can use AWS Secrets Manager or manage your master user credentials.

☒ **Managed in AWS Secrets Manager - most secure**
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☐ **Self managed**
Create your own password or have RDS create a password that you manage.

Info If you manage the master user credentials in AWS Secrets Manager, additional charges apply. See [AWS Secrets Manager pricing](#). Additionally, some RDS features aren't supported. See [limitations here](#).

Amazon S3

Buckets

General purpose buckets | All AWS Regions | Directory buckets

General purpose buckets (1) Info

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
expense-tracker-receipts-2026-287666000191-us-east-1-an	US East (N. Virginia) us-east-1	June 28, 2026, 16:04:37 (UTC+05:30)

Account snapshot Info View dashboard

Updated daily

Storage Lens provides visibility into storage usage and activity trends.

External access summary Info

Updated daily

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

Amazon S3 > Buckets > expense-tracker-receipts-2026-287666000191-us-east-1-an

Objects (7)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions

Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
hospital.jpg	jpg	June 29, 2026, 18:29:32 (UTC+05:30)	76.8 KB	Standard
hospital1.jpg	jpg	June 29, 2026, 18:52:39 (UTC+05:30)	291.0 KB	Standard
hospital2.jpg	jpg	June 29, 2026, 19:08:51 (UTC+05:30)	7.4 KB	Standard

Lambda > Functions

Functions (2/7) Last fetched 7/12/2026, 5:21:36 PM Actions Create function

Functions table allows you to query function details. Use Ctrl + K to start a in-console resource search.

Search by attributes or search by keyword

Function name	Description	Package type	Runtime	Type	Last modified
ExpenseSummaryAPI	-	Zip	Python 3.14	Standard	2 weeks ago
ModLabRole	updates LabRole to allow it to assume itself	Zip	Python 3.10	Standard	2 months ago
ExpenseTrackerOCR	-	Zip	Python 3.14	Standard	1 week ago
RedshiftOverwatch	Deletes Redshift Cluster if the count is more than 2.	Zip	Python 3.10	Standard	2 months ago
MainMonitoringFunction	-	Zip	Python 3.10	Standard	2 months ago

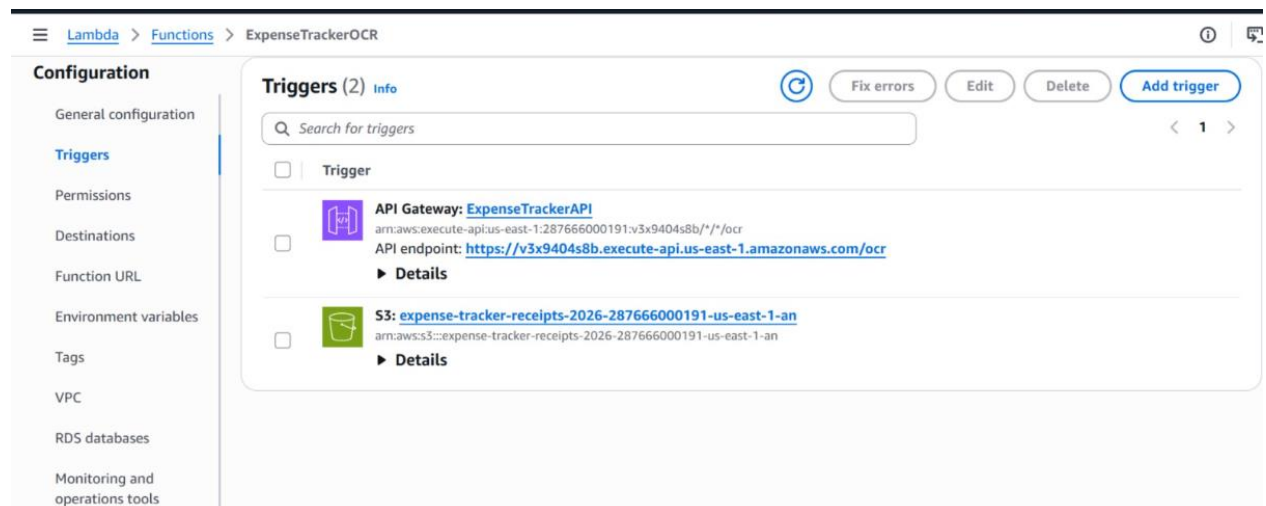
4.2 Receipt Storage Layer – Amazon S3

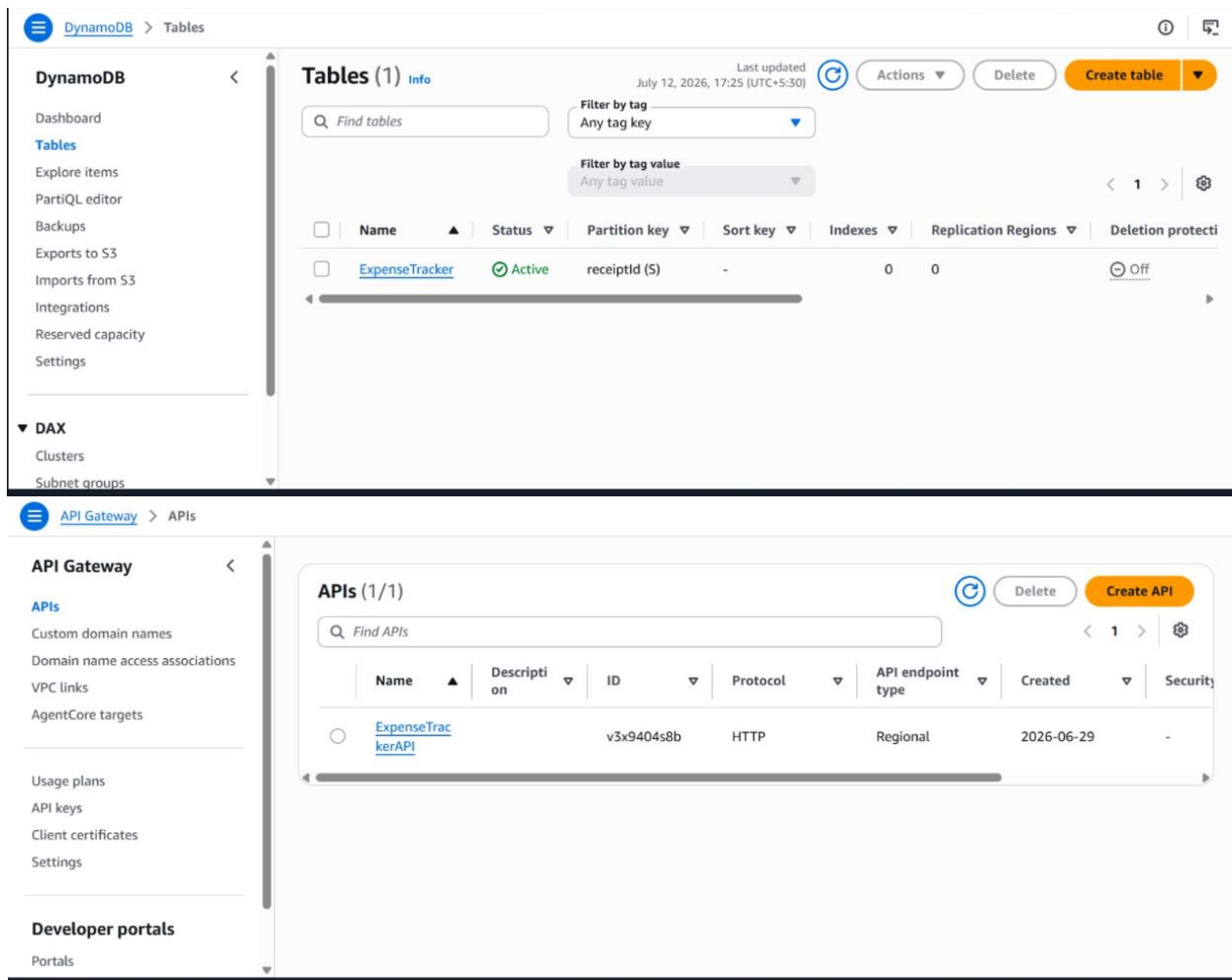
The process begins when a user uploads a receipt image through the application. The uploaded image is stored securely in an **Amazon S3 bucket**, which serves as the cloud storage service.

Amazon S3 is configured with **Event Notifications**, allowing it to automatically trigger an AWS Lambda function whenever a new receipt image is uploaded.

Key Features

- Secure cloud storage
- High durability
- Automatic event triggering
- Supports JPG, PNG, and other image formats
- Highly scalable





4.3 Processing Layer – AWS Lambda

Once the receipt is uploaded into Amazon S3, an **AWS Lambda** function is automatically triggered.

The Lambda function performs the following operations:

- Retrieves the uploaded receipt image
- Invokes Amazon Textract
- Receives OCR results
- Processes and validates extracted data

- Formats expense information
- Stores the processed records in DynamoDB

Using Lambda enables completely serverless execution without managing backend servers.

4.4 OCR Processing – Amazon Textract

Amazon Textract is the core Artificial Intelligence service used in this project.

Textract performs Optical Character Recognition (OCR) on the uploaded receipt image and extracts important expense information automatically.

The extracted information includes:

- Merchant Name
- Purchase Date
- Total Amount
- Tax Details
- Receipt Number (if available)
- Additional expense information

Compared to traditional OCR solutions, Amazon Textract provides higher accuracy and supports structured document analysis.

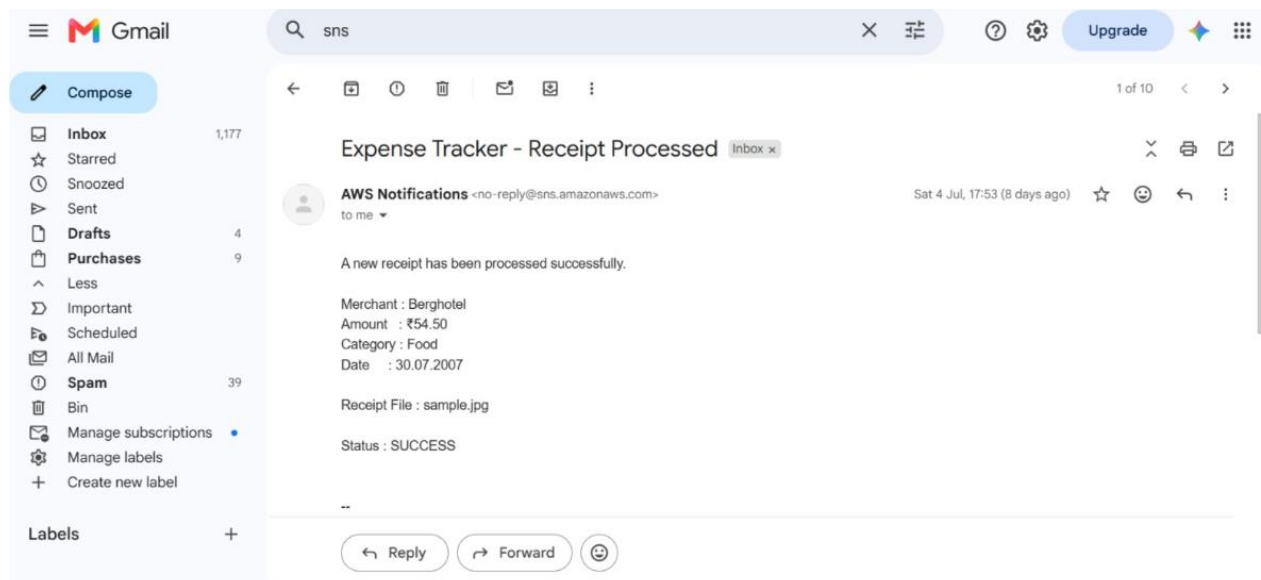
4.5 Database Layer – Amazon DynamoDB

After OCR processing, the extracted expense information is stored in **Amazon DynamoDB**.

Each expense record contains information such as:

- Expense ID
- Merchant Name
- Purchase Date
- Total Amount
- Receipt Details
- Receipt Image URL
- Upload Timestamp

The database enables fast retrieval of expense records and supports searching, filtering, and expense history management.





Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:us-east-1:287666000191:ExpenseTrackerNotifications:784dc8b1-ff02-462e-83d7-41584a35b82e

If it was not your intention to subscribe, [click here to unsubscribe](#).

Advantages

- Fully managed NoSQL database
- Automatic scaling
- High-speed read/write operations
- Low latency

4.6 API Layer – Amazon API Gateway

Amazon API Gateway acts as the communication bridge between the frontend application and AWS backend services.

It provides secure REST APIs that allow users to:

- Upload receipt information
- Retrieve expense history

- Search expenses
- Display dashboard data

API Gateway securely routes all client requests to the appropriate Lambda functions.

4.7 Security Layer – AWS IAM

AWS Identity and Access Management (IAM) is responsible for securing communication among AWS services.

IAM roles ensure:

- Lambda can access S3
- Lambda can invoke Textract
- Lambda can write to DynamoDB
- API Gateway can invoke Lambda

Least-privilege access policies are implemented to improve overall security.

4.8 Monitoring and Logging – Amazon CloudWatch

Amazon CloudWatch continuously monitors the application's performance.

CloudWatch is used for:

- Monitoring Lambda executions
- Viewing application logs

- Error tracking
- Performance analysis
- Debugging failures

It helps developers quickly identify and resolve issues within the application.

4.9 System Workflow

The Smart OCR Expense Tracker follows a serverless, event-driven workflow using Amazon Web Services (AWS). The system automates the process of extracting expense details from receipt images and securely stores them for future access. The complete workflow is described below.

Step 1: Receipt Upload

The user uploads a receipt image through the web application. The uploaded receipt may be a shopping bill, restaurant invoice, fuel receipt, grocery bill, or any other expense receipt. The image is stored securely in an Amazon S3 bucket.

Step 2: Event Trigger

Whenever a new receipt image is uploaded to the S3 bucket, an ObjectCreated Event is automatically generated. This event triggers the ExpenseTrackerOCR AWS Lambda function without requiring any manual intervention.

Step 3: OCR Processing

The Lambda function sends the uploaded receipt image to Amazon Textract. Textract performs Optical Character Recognition (OCR) to identify and extract important information such as:

- Merchant Name
- Purchase Date
- Total Amount

- Tax Amount
- Receipt Number
- Other available receipt details

Step 4: Data Processing

After receiving the OCR output from Amazon Textract, the Lambda function processes and validates the extracted information. The data is converted into a structured format suitable for database storage.

Step 5: Data Storage

The processed expense details are stored in the ExpenseTracker table in Amazon DynamoDB. This allows the application to efficiently retrieve and manage expense records whenever required.

Step 6: Email Notification

After successfully storing the expense information, the Lambda function publishes a notification to Amazon SNS. Amazon SNS sends an email to the registered user confirming that the receipt has been processed successfully.

Step 7: Expense Summary Request

When the user wants to view expense records or generate an expense summary, the request is sent to Amazon API Gateway. API Gateway forwards the request to the ExpenseSummaryAPI AWS Lambda function.

Step 8: Retrieve Expense Data

The ExpenseSummaryAPI Lambda function retrieves the required expense records from Amazon DynamoDB and prepares the response in JSON format.

Step 9: Display Expense Summary

The processed expense data is returned to the frontend application, where users can view their expense history, search previous receipts, and monitor their overall spending.

Step 10: Monitoring and Logging

Throughout the entire workflow, Amazon CloudWatch continuously records Lambda execution logs, monitors system performance, and captures errors for troubleshooting and maintenance.

4.10 Architecture Advantages

The proposed architecture offers several benefits:

- Fully serverless architecture
 - Automatic OCR-based expense extraction
 - Scalable cloud infrastructure
 - Secure data storage
 - High availability and reliability
 - Minimal infrastructure management
 - Faster receipt processing
 - Reduced manual effort
 - Improved data accuracy
 - Cost-effective solution using AWS managed services
-

5. IMPLEMENTATION

The implementation of the **Smart OCR Expense Tracker** focuses on automating expense management using a serverless architecture on Amazon Web Services (AWS). The application integrates multiple AWS services, including Amazon S3, AWS Lambda, Amazon Textract, Amazon DynamoDB, Amazon API Gateway, Amazon SNS, and Amazon CloudWatch, to provide an efficient and scalable expense tracking solution. The implementation process is divided into several stages as described below.

5.1 Receipt Image Upload

The first step in the implementation is uploading the receipt image.

The user uploads a receipt image through the web application. The uploaded image may contain shopping bills, restaurant receipts, fuel bills, hotel invoices, or any other purchase receipts. The receipt image is stored securely in an **Amazon S3 bucket**, which serves as the storage layer of the application.

Amazon S3 is configured with an **ObjectCreated Event Notification**, allowing the application to automatically trigger backend processing whenever a new receipt image is uploaded.

Figure 5.1: Amazon S3 Bucket Configuration

5.2 Lambda Function Configuration

The backend processing is implemented using **AWS Lambda**, eliminating the need for dedicated servers.

The project consists of two Lambda functions:

ExpenseTrackerOCR

This function is automatically triggered whenever a receipt image is uploaded into Amazon S3.

The function performs the following tasks:

- Reads the uploaded receipt image
- Invokes Amazon Textract
- Receives OCR output
- Processes extracted information
- Stores expense records in DynamoDB
- Sends email notification through SNS

Figure 5.2: ExpenseTrackerOCR Lambda Function

ExpenseSummaryAPI

The ExpenseSummaryAPI Lambda function is responsible for generating expense summaries.

It performs the following operations:

- Receives requests from API Gateway
- Retrieves expense records from DynamoDB
- Calculates expense summaries
- Returns the response to the frontend application

Figure 5.3: ExpenseSummaryAPI Lambda Function

5.3 OCR Integration using Amazon Textract

Amazon Textract is integrated with the ExpenseTrackerOCR Lambda function to perform Optical Character Recognition (OCR).

After the receipt image is uploaded, Lambda sends the image to Amazon Textract.

Textract automatically extracts important information, including:

- Merchant Name
- Purchase Date
- Total Amount
- Tax Amount
- Receipt Number
- Purchased Items (if available)

The extracted information is returned in JSON format, which is further processed by the Lambda function.

Figure 5.4: Amazon Textract OCR Processing

5.4 Expense Data Processing

After receiving the OCR response, the Lambda function processes the extracted data.

The implementation performs:

- Data validation
- Text cleaning
- JSON parsing
- Expense formatting
- Duplicate checking (if applicable)

The processed expense information is converted into a structured format before database storage.

5.5 Database Implementation

Amazon DynamoDB is used as the primary NoSQL database.

The project uses an **ExpenseTracker** table for storing expense records.

Each record contains:

- Receipt ID
- Merchant Name
- Purchase Date
- Total Amount
- Receipt Details
- Timestamp

DynamoDB provides low-latency access and automatic scalability.

Figure 5.5: ExpenseTracker DynamoDB Table

5.6 API Implementation

Amazon API Gateway is configured to expose REST API endpoints.

The API Gateway forwards HTTP requests to the ExpenseSummaryAPI Lambda function.

The implemented API supports:

- Expense Summary

- Expense History
- Expense Retrieval

The API returns responses in JSON format.

Figure 5.6: Amazon API Gateway Configuration

5.7 Email Notification

Amazon SNS is integrated with the ExpenseTrackerOCR Lambda function.

After successfully processing a receipt, the application publishes a notification to an SNS Topic.

Amazon SNS automatically sends an email containing the receipt processing status to the registered user.

This confirms that the expense information has been successfully stored.

Figure 5.7: Amazon SNS Notification

5.8 Monitoring and Logging

The application uses Amazon CloudWatch for monitoring and debugging.

CloudWatch automatically records:

- Lambda execution logs
- Function duration
- Errors
- Invocation count

- Memory utilization

These logs help developers monitor application performance and troubleshoot issues efficiently.

Figure 5.8: Amazon CloudWatch Logs

5.9 Security Implementation

AWS IAM Roles are configured to provide secure communication among AWS services.

The implemented permissions include:

- S3 Read Permission
- Textract Access Permission
- DynamoDB Read/Write Permission
- SNS Publish Permission
- CloudWatch Logging Permission

The principle of **Least Privilege** is followed to improve security.

6. RESULTS

The **Smart OCR Expense Tracker** was successfully implemented using a serverless architecture on **Amazon Web Services (AWS)**. The application achieved its primary objective of automating expense management by extracting important information from receipt images using **Amazon Textract**. The system successfully processed uploaded receipts, extracted expense details, stored the information securely, and provided users with an efficient platform to manage and analyze their expenses.

During testing, users uploaded different types of receipt images, including shopping bills, restaurant receipts, grocery bills, and fuel bills. Every uploaded receipt was successfully stored in the configured **Amazon S3 bucket**. The upload event automatically triggered the **ExpenseTrackerOCR AWS Lambda** function without any manual intervention. The Lambda function processed the uploaded image and invoked **Amazon Textract** to perform Optical Character Recognition (OCR). The extracted information, including the merchant name, purchase date, total amount, and other available receipt details, was successfully processed and converted into a structured format.

After successful processing, the expense information was stored in the **ExpenseTracker DynamoDB** table. The stored data was then retrieved through the **ExpenseSummaryAPI Lambda** function using **Amazon API Gateway**, enabling users to view their expense history and summaries. The application also successfully generated email notifications through **Amazon SNS**, confirming that receipt processing had been completed successfully. Throughout the execution, **Amazon CloudWatch** continuously monitored the Lambda functions and generated execution logs, ensuring reliable monitoring and easier debugging.

The implemented serverless architecture demonstrated excellent scalability, reliability, and efficiency. Since all processing was event-driven, the application automatically handled receipt uploads without requiring dedicated servers or manual intervention. The use of managed AWS services reduced infrastructure complexity while providing secure and cost-effective expense management.

6.2 Output Analysis

The application successfully extracted expense information from various receipt formats with good accuracy. Amazon Textract effectively recognized printed text from uploaded receipt images and returned structured data that was processed by the Lambda function. The extracted information was accurately stored in Amazon DynamoDB, allowing users to retrieve their expense history whenever required. The API Gateway and ExpenseSummaryAPI Lambda function provided quick responses for expense summary requests, while Amazon SNS delivered confirmation emails immediately after successful receipt processing.

The system also maintained detailed execution logs using Amazon CloudWatch, making it easy to monitor application performance and identify any runtime issues. Overall, the application performed reliably during testing and successfully completed the complete workflow from receipt upload to expense data retrieval.

6.3 Output Screens

The successful implementation of the Smart OCR Expense Tracker is demonstrated through the following in this report.

- Receipt Image Uploaded to Amazon S3
- ExpenseTrackerOCR Lambda Function
- Amazon Textract OCR Processing
- ExpenseTracker DynamoDB Table
- ExpenseSummaryAPI Lambda Function
- Amazon API Gateway Configuration
- Amazon SNS Email Notification
- Amazon CloudWatch Execution Logs

6.4 Discussion

The Smart OCR Expense Tracker successfully fulfilled all the objectives defined at the beginning of the project. The integration of Amazon S3, AWS Lambda, Amazon Textract, Amazon DynamoDB, API Gateway, Amazon SNS, and CloudWatch enabled the development of a completely serverless expense management system. The application significantly reduced manual effort by automatically extracting expense information from receipt images and securely storing it in the cloud. The implemented solution proved to be scalable, reliable, and efficient, making it suitable for both personal and small business expense management. The project also demonstrated the practical application of cloud computing, OCR technology, and serverless architecture in solving a real-world financial management problem.

7.CONCLUSION

The **Smart OCR Expense Tracker** was successfully designed and implemented as a serverless cloud-based application using **Amazon Web Services (AWS)**. The project effectively automates the process of extracting expense information from receipt images, eliminating the need for manual data entry. By integrating **Amazon S3**, **AWS Lambda**, **Amazon Textract**, **Amazon DynamoDB**, **Amazon API Gateway**, **Amazon SNS**, and **Amazon CloudWatch**, the application provides a secure, scalable, and efficient solution for expense management.

The system successfully stores uploaded receipt images in Amazon S3, automatically extracts relevant information using Amazon Textract, processes the extracted data through AWS Lambda, and stores structured expense records in Amazon DynamoDB. Users can retrieve their expense history and summaries through API Gateway, while Amazon SNS provides email notifications confirming successful receipt processing. Amazon CloudWatch continuously monitors the application's performance and records execution logs, ensuring reliability and simplifying maintenance.

The serverless architecture adopted in this project eliminates the need for server management, reduces operational costs, and enables automatic scaling based on application demand. The event-driven workflow ensures fast and reliable processing of receipt images while improving data accuracy and reducing human errors associated with manual expense tracking.

Overall, the **Smart OCR Expense Tracker** successfully achieved all the objectives of the project by providing an intelligent and automated expense management system. The project demonstrates the practical application of cloud computing, Optical Character Recognition (OCR), and serverless technologies in solving a real-world problem. It also highlights the benefits of AWS managed services in building scalable, secure, and cost-effective applications.

Although the current implementation provides the essential features required for expense tracking, the system can be further enhanced by incorporating advanced capabilities such as AI-based expense categorization, budget planning, spending analytics, multi-user authentication using Amazon Cognito, mobile application support, and report generation in PDF or CSV formats. These enhancements would further improve the usability and functionality of the application, making it suitable for both personal and enterprise-level expense management.